12 mars 2026

Journée d'étude du Consortium Musica*
IR* Huma-Num

modéliser les données de la recherche avec le CIDOC CRM

pourquoi un standard + problèmes d'ingénierie logicielle

📡 thomas.bottini@cnrs.fr

Institut de Recherche en Musicologie UMR 8223

[la question qui nous réunit]

Comment faire tenir les données de la recherche dans le temps ?

- 📜 Comment s'assurer qu'on puisse toujours les voir, les comprendre, les modifier ?
- 🔭 Comment capter et transmettre les « traditions interprétatives » sans les amoindrir ?
- 🎓 Comment fonctionner avec des moyens 👥/💻/💶 limités ?
- 🌐 Comment favoriser les liens avec d'autres systèmes d'information ?
- 💾 À quels 🧺 confier ses 🥚 ?

modéliser sous contraintes de ressources

- La modélisation des données nécessite une réflexion conjointe entre chercheurs et ingénieurs.
  - Ce dialogue a une fonction maïeutique, il doit confronter le chercheur ou la chercheuse à des cas limites pour l'amener à mieux comprendre ses objets d'étude.
  - Le travail d'explicitation, de modélisation, des données a une fonction heuristique : aider à révéler la structure interne des sources et des phénomènes étudiés.
- Or, souvent, la modélisation est perçue comme une étape purement technique par le client et est renvoyée à la compétence du prestataire.
- Les ressources d'ingénierie sont trop maigres, ce niveau de dialogue est rare.

De plus, l'époque est au FAIR et au LOD⭐⭐⭐⭐⭐ (des notions relativement spécifiques à notre contexte professionnel). L'ouverture des données de la recherche, leur interopérabilité et leur mise en relation avec des sources de données tierces nécessitent une réflexion technique, méthodologique et scientifique très en amont dans le projet. Après, c'est souvent trop tard.

ce qu'on cherche à palier avec les consortiums

- La FAIRisation des données suppose une dynamique informationnelle intellectuelle et technique entre les projets de BDD.
- Pour la bâtir, un réseau d'acteurs et d'actrices est nécessaire, mais :
  - Il faut une complémentarité recherche🔬/ingénierie⚙️/SIB📚 car ces connaissances sont très abstraites et difficiles à saisir.
  - Les chercheurs et chercheuses devant piloter de tels projet manquent d'informations claires sur les conséquences scientifiques et méthodologiques des technologies disponibles pour modéliser les informations scientifiques.
  - Les profils techniques sont recrutés sur des contrats courts.
  - Les prestataires n'ont pas d'intérêt à s'inscrire dans les réseaux HN.
- Conséquemment, les connaissances d'ingénierie spécifiques à la modélisation des données de la discipline sont peu capitalisées ; chaque nouveau développement peine à bénéficier de l'expérience méthodologique et conceptuelle acquise informellement au fil des projets passés.

ENCODER

MODÉLISER

SAISIR

EXPLORER

PÉRENNISER

ENCODER

quelques problèmes potentiels avec les sgbdr

- Un modèle relationnel est souvent conçu ici & maintenant : il formalise et matérialise une tâche d'analyse d'une pratique de recherche spécifique. Le risque est de « réinventer » la roue, sur le plan conceptuel, à chaque réitération de cette démarche d'analyse. Pourtant, il existe des similarités entre les pratiques de recherche en SHS liées aux données numériques.
- La sémantique d'un modèle relationnel ne peut être techniquement partagée, ce qui peut poser des problèmes de lecture et de compréhension plus tard.
- L'accès aux données par un système tiers peut être malaisé. Souvent, on doit développer une API, qui est déterminée par le cahier des charges du projet, alors qu'on sait qu'on ne peut anticiper tous les usages futurs des données (c'est l'idée même des LOD). C'est aussi un point de tension dans le dialogue avec le prestataire : tout changement apporté au modèle vu comme minime par le client peut avoir d'importantes conséquences sur le développement.
- Tout ceci amène à parler d'architectures « en silo ».

pervasivité des sgbdr

- Le panorama technologiques des outils permettant de construire rapidement des applications Web de saisie collaborative de données est dominé par les systèmes reposant sur les SGBDR :
  - CMS & assimilés (Wagtail, Drupal, Omeka, Heurist)
  - systèmes « no-code »
  - frameworks full-stack qui existent dans tous les langages de programmation
- Les SGBDR, et avec eux, la méthode Merise et le SQL, sont enseignés partout, à tous les niveaux. Les compétences d'ingénierie sont donc très répandues.

le web sémantique

- Promesse d'une base de données à l'échelle du Web. Le Web initial (Tim Berners Lee, 1991) était un Web de documents liés (hypertexte), le Web sémantique est un Web de données liées, chacune étant identifiée par une URL.
- Toute information s'exprime sous la forme d'un triplet sujet/prédicat/objet dans un langage de description qui est le RDF.

- Connectés, ces triplets RDF forment un graphe. Idée de modèle ouvert.
- Chaque prédicat est également identifié par une URL. Cela permet de toujours savoir de quoi on parle quand on dit, par exemple : « Titre ».
- Tout graph RDF est interrogeable en SPARQL (+ standardisé que SQL).
- C'est le milieu technique idéal pour nos préoccupations : LOD, FAIR, etc.

données relationnelles vs graphe rdf

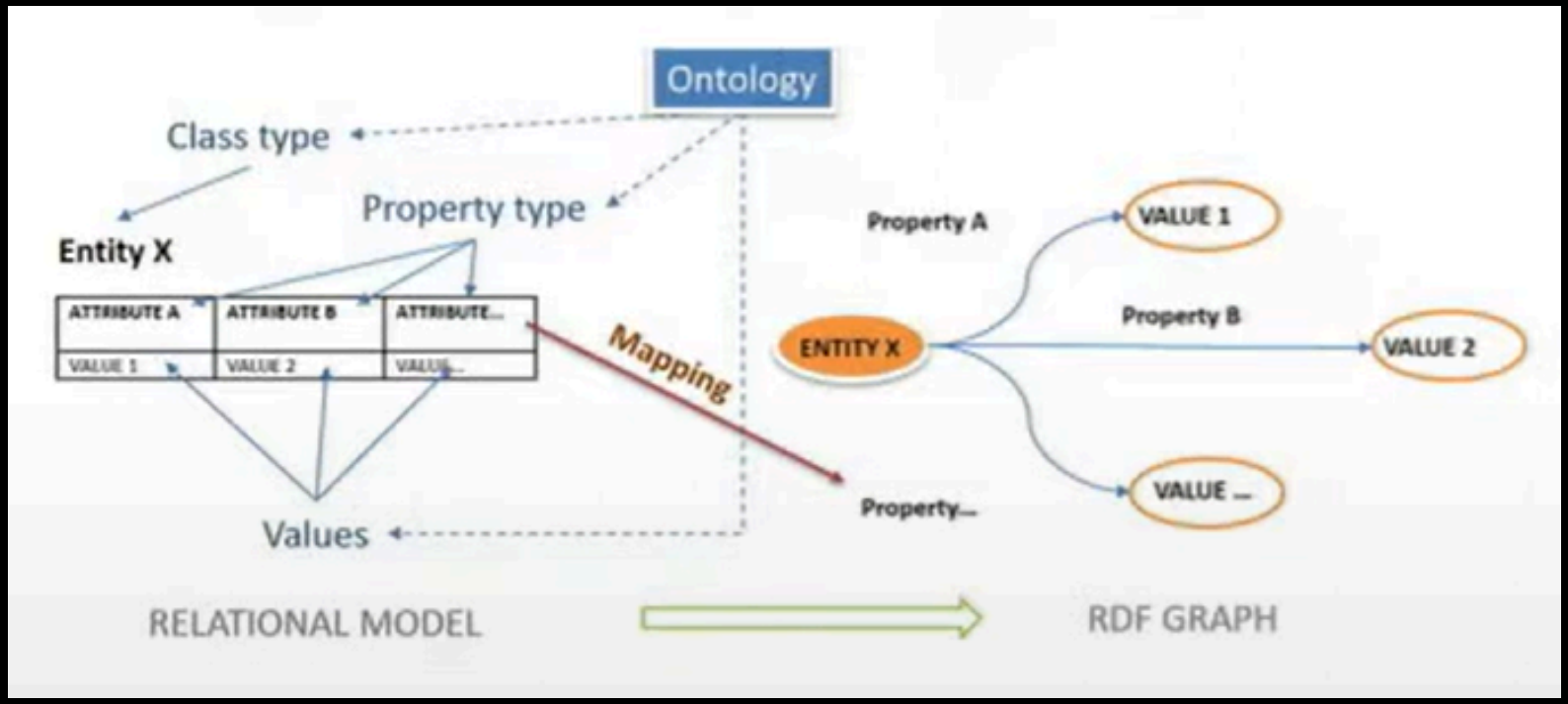


Fig. 1. – Corago in LOD - Seminar by Angelo Pompilio and Paolo Bonora, Digital Humanities and Digital Knowledge, Università di Bologna, 2017.

MODÉLISER

une ontologie : quoi, pourquoi ?

- Formalisation d'un modèle conceptuel pour un domaine donné, contenant des classes et des propriétés.
- Utiliser les classes et les propriétés d'une ontologie confère ainsi une sémantique partagée aux données.
- Permet de capitaliser des connaissances de modélisation d'un projet à l'autre.
- Le Web sémantique s'accompagne de standards utiles pour nos métiers : SKOS pour les thésaurus, DCMI pour les métadonnées basiques, DOREMUS pour la musique écrite/éditée/jouée/diffusée, LRMOO pour l'information bibliographique, PROV-O pour la provenance de l'information, et bien sûr le CIDOC CRM.
- Bien des projets de recherche alliant informatique et SHS se donnent pour mission de produire une nouvelle ontologie pour couvrir un besoin spécifique. Nous défendons la thèse opposée : tenter de tout modéliser avec le CIDOC CRM.

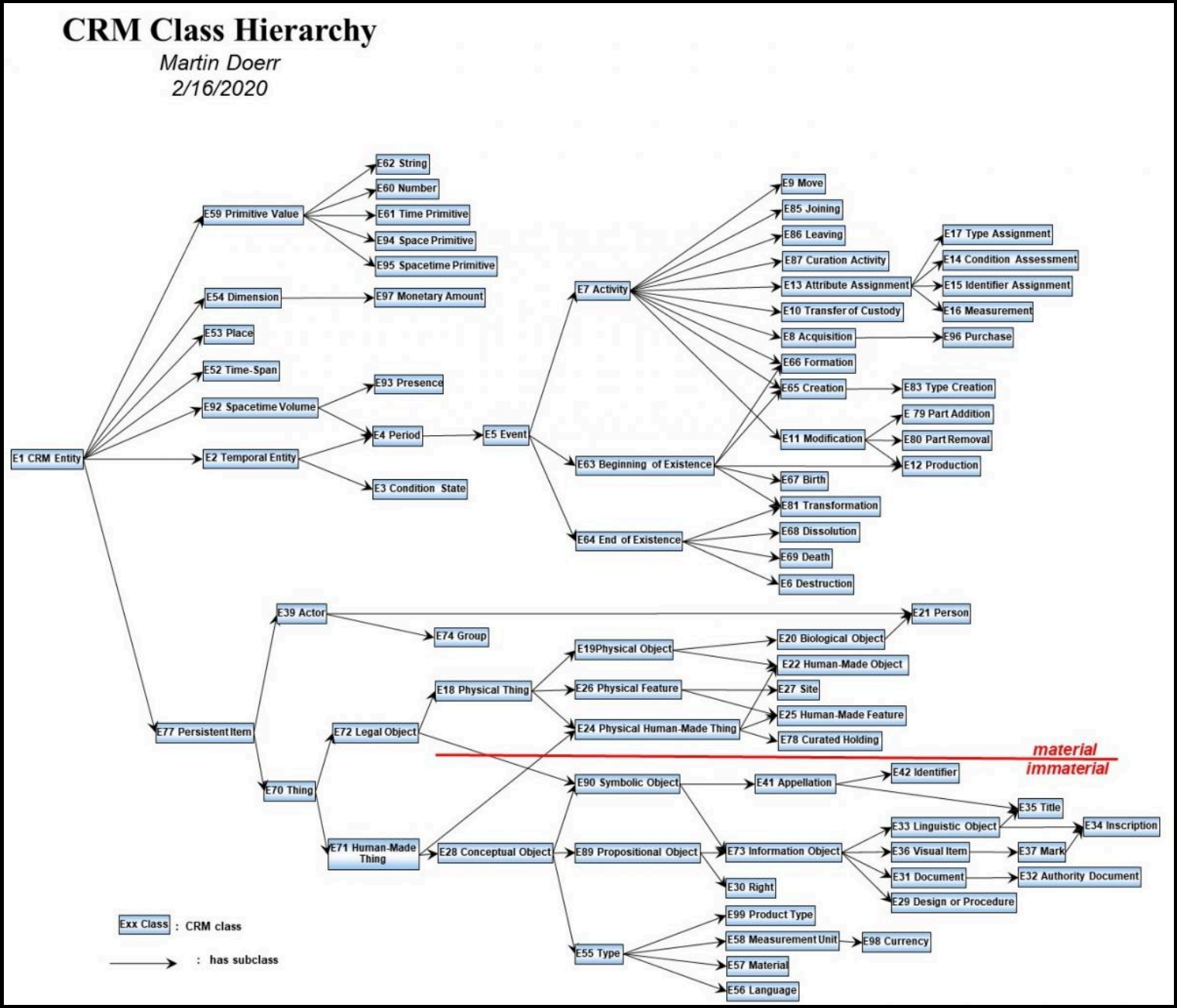
une ontologie unique : motivations sur le plan de l'ingénierie logicielle

- Le recours à un standard pour représenter les données permet mieux structurer le développement informatique.
- Par exemple, des standards métier établis dans nos disciplines comme IIIF, MEI ou TEI rendent possible le fleurissement d'initiatives logicielles indépendantes.
- ⚙️ MODULARISATION
  - Des composants peuvent être développés en vue d'être réutilisés dans différents projets, ils savent toujours à quoi ressemblent leurs données.
- 🤝 COMPRÉHENSION
  - Un partage de la compréhension du modèle entre les acteurs participants au développement des différents composants d'un système d'information facilite leurs interactions, et limite les méprises sur les spécifications fonctionnelles.
  - Imaginons un monde où développeurs, utilisateurs et concepteurs graphiques partagent une compréhension profonde d'un modèle unique.
- Avantages méthodologiques, scientifiques, techniques, économiques.

laquelle ? le cidoc crm !

- Ontologie qui documente le patrimoine matériel et immatériel ainsi que les processus de production de connaissances à son propos (sources, connaissances, faits sociaux, concepts, objets matériels, idées dénotées ou connotées, contexte de production des connaissances, etc.).
- Communauté large et établie. Venant du monde des musées, elle est désormais utilisée dans tous les domaines des HN
- Ontologie centrée événement.
- Informations bibliographiques avec LRMoo (œuvres, expressions, manifestation, item).

hiérarchie des classes du cidoc crm



opérations critiques & patterns de base

- Nommer
- Identifier
- Typer, catégoriser, indexer, classifier (avec des concepts métier)
- Dater
- Établir le fait qu'une information fait référence à une entité (dénotation/connotation)
- Contextualiser, caractériser le contexte de production d'une chose ou d'une idée
- Structurer (entités matérielles et informationnelles)
- Annoter (plus généralement : signer tout apport de connaissance sur une entité)
- Établir la participation d'un acteur à un phénomène temporel ou social
- Décrire l'évolution dans le temps et l'espace d'une entité
- Décrire l'influence d'une chose ou d'une expérience sur une activité.

=> Tous ces « patterns » de modélisation permettent de capter les spécificités de nos pratiques scientifiques.

s'y plonger

- Introduction to the basic concepts (une dizaine de pages)
- Modèle
  - centré événements
  - prenant en charge les relations spatio-temporelles
  - avec notation extensive du temps

⚠️ Le CRM n'est pas un système de classification. Le fait de dire qu'une entité de votre corpus relève d'une certain classe du CRM lui confère un statut structurel/fonctionnel, et ne dit rien de sa teneur scientifique.

Par exemple, si on veut dire qu'une source textuelle (instance de la classe `crm:E33_Linguistic_Object`) est un traité de théorie musicale, il faut créer un concept « Traité de théorie musicale » et l'utiliser pour typer la ressource.

Les concepts issus du métier sont des instances de la classe `crm:E55_Type`, qui est équivalente à `skos:Concept`. Le CRM s'utilise naturellement en conjonction avec vos thésaurus pour sémantiser vos entités. À nouveau, c'est une ontologie générique.

SAISIR

ça se complique, sur le plan technique

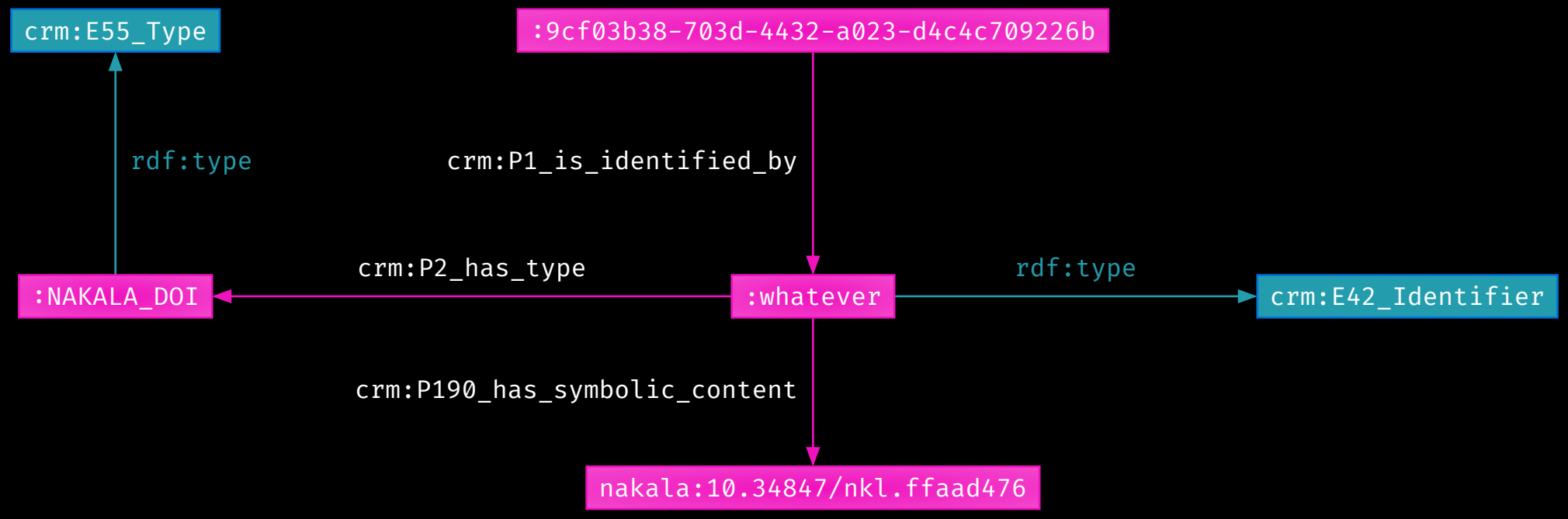
- Un graphe ouvert est plus difficile à éditer que des données relationnelles, qui s'éditent naturellement en série dans un tableau ou avec des formulaires contraints.
- Les patterns fondamentaux du CRM (pour nommer, typer, dater, annoter, contextualiser...) induisent beaucoup de sous-entités. Nous sommes loins du modèle ligne/colonne/cellule. Des vocabulaires de schémas comme `DCMI Metadata Terms` ou `schema.org` sont plus simples (mais leur expressivité est loin d'égaler celle du CRM).
- Rien ne peut battre un tableur pour l'édition d'items similaires rassemblés dans une collection. Au sein de nos communautés, Excel n'est pas qu'un choix par défaut.

exemple : conférer un identifiant à une ressource

	UUID	Nakala DOI
1	9cf03b38-703d-4432-a023-d4c4c709226b	https://nakala.fr/10.34847/nkl.ffaad476

VS

```
@base <http://data-iremus.huma-num.fr/id/> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix crm: <http://www.cidoc-crm.org/cidoc-crm/> .
@prefix nakala: <https://nakala.fr/> .
```



ça se complique, sur le plan conceptuel

- Le CRM est très expressif : il aide à ne pas réduire ni trahir les productions analytiques des chercheurs et des chercheuses. [A]
- Son double caractère abstrait & générique le rend apte à modéliser un large éventail de données convenant à un large éventail de pratiques. Certes, le CRM n'est pas optimal pour chaque scénario, mais on peut se demander ce qu'il n'est pas capable de modéliser.
- Mais le CRM est complexe à comprendre et à mettre en œuvre…
  - Il existe parfois plusieurs manières de modéliser une situation avec les classes de base.
  - En tant qu'ontologie abstraite et générique, sa structure représentée par ses classes et propriétés fait écran avec la compréhension spontanée que l'on pourrait avoir de nos données. Certains chemins, quoique très précis sur le plan de la modélisation, sont alambiqués. (opinion : c'est le prix de la proposition [A])
- Conséquemment, chaque collectif s'approprie l'ontologie selon ses pratiques, en ne retenant que certaines classes et propriétés et en favorisant certains patterns de modélisation.
- => Une interface d'édition générique de données CRM n'a pas vraiment de sens. L'ontologie est peut-être trop large, trop peu focalisée… Des solutions ayant une approche plus « locale » seraient-elles à privilégier ?

notre objectif

[nous = le consortium Musica*]

💡 Nous aimerions une système d'information générant des données CRM :

[1] qui s'adapte aux conditions de saisie spécifiques à chaque pratique scientifique

[2] qui n'ajoute pas de charge mentale aux chercheurs et aux chercheuses, ni n'impose de maîtriser une complexité formelle trop éloignée des objets de la recherche

[3] mais qui tire parti des vertus heuristiques du CRM (on est là pour ça après tout !)

[4] et qui n'impose pas, sur le plan de la gestion de projet et de la gestion de nos ressources de développement, de développer un outil de saisie générique de données CRM (car c'est coûteux et surtout inadéquat, comme on l'a vu)

[5] qui facilite les tâches techniques : déploiement, maintenance, extraction des données structurées, paramétrage des règles de mapping, génération des données RDF-CRM

[6] et en plus qui ajoute une couche de structure et de normalisation par rapport aux pratiques de mapping avec des outils dédiés

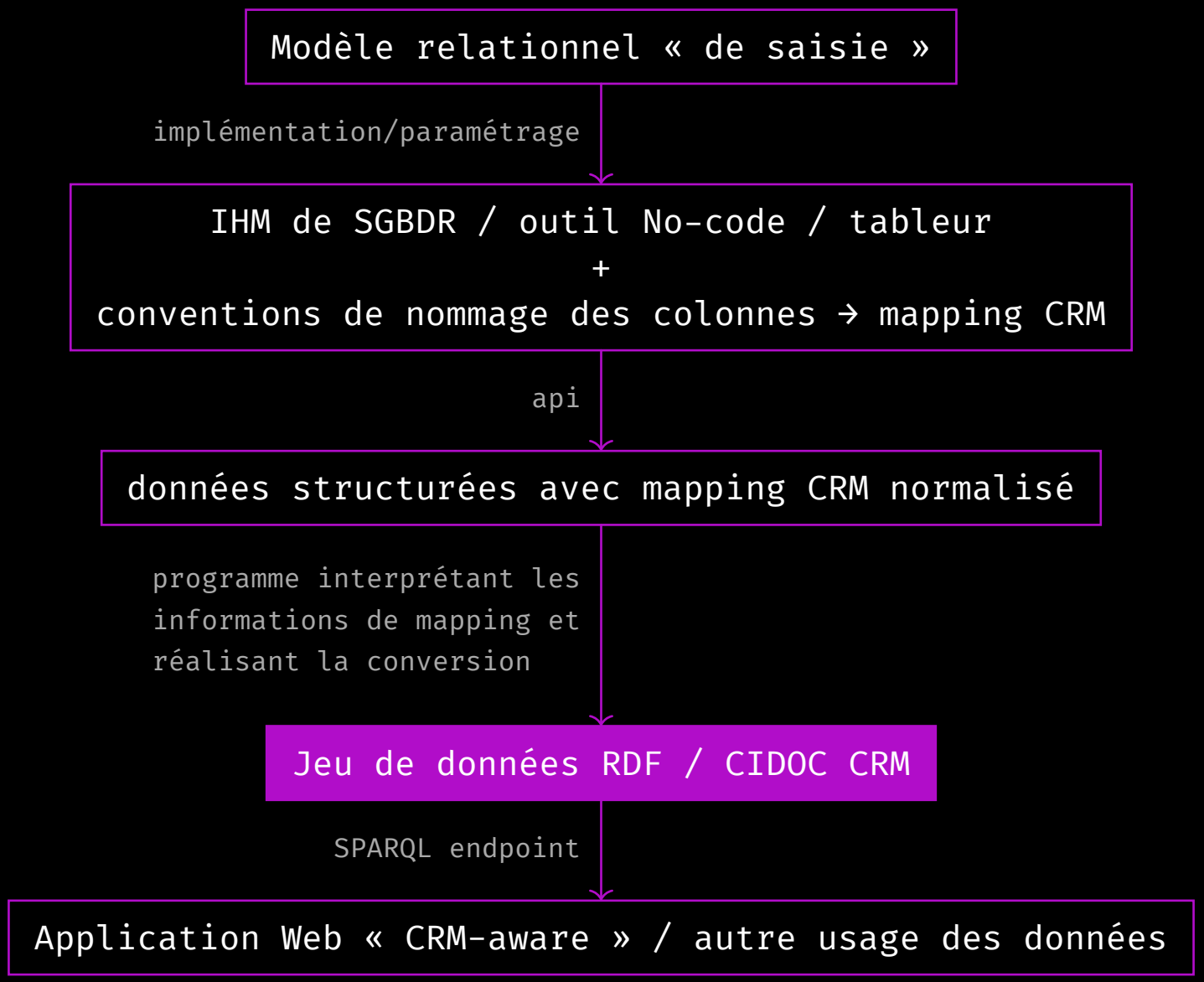
les sgbdr à leur juste place

- Pour être au plus proche des pratiques de saisie [1][2] et de la culture technique dominante [5], nous préconisons le recours à un outil de saisie de données existant reposant sur un SGBDR, libre et ergonomique.
- Par exemple, un candidat de la constellation « No-code ». Un SGBDR permet d'aller bien plus loin qu'une feuille Excel. Les outils « No-code » contemporains se rapprochent de cette ergonomie « tabulaire ». Avec la généralisation de ces outils, il y a moins de raisons de recourir à un tableur offline. [4]
- Dans cette perspective, le modèle relationnel devrait être créé pour répondre aux attendus ergonomiques d'un projet de recherche spécifique. Sa structure devrait permettre de générer des données CRM par la suite, mais il ne serait ici qu'un modèle de saisie, et non un modèle destiné à l'échange ou à la tenue dans le temps des données. [1][5]

vers un mapping normalisé

- Lors, par exemple, d'une reprise de l'existant en vue d'une conversion en données CRM, on définit des règles de mapping spécifiques.
- Nous aimerions réduire au minimum les ajustements ad-hoc pour chaque nouveau projet.
- Nous aimerions ainsi définir un standard de mapping entre données relationnelles et données RDF/CIDOC CRM. Ce serait une manière normalisée & reproductible de faire correspondre une structure relationnelle/tabulaire à un graph RDF/CIDOC CRM. [6]
- Ceci pourrait être simplement réalisé en normalisant l'implémentation des structures relationnelles (nom des tables, convention de nommage des colonnes, clefs primaires, clefs étrangères, contraintes d'intégrité référentielles) et en s'appuyant sur des conventions de représentation des patterns de modélisation du CRM.
- Il faut identifier ces patterns, et pour chacun d'entre eux, imaginer une implémentation relationnelle réalisant le compromis optimum entre ergonomie de saisie et expressivité permise par le CRM (cf. exemple des rôles & événements de création pour une œuvre). [3]

synthèse


Modèle relationnel « de saisie »
implémentation/paramétrage
IHM de SGBDR / outil No-code / tableur
+
conventions de nommage des colonnes → mapping CRM
api
données structurées avec mapping CRM normalisé
programme interprétant les informations de mapping et réalisant la conversion
Jeu de données RDF / CIDOC CRM
SPARQL endpoint
Application Web « CRM-aware » / autre usage des données

🖖

grist

- Pourquoi Grist ?
  - https://lasuite.numerique.gouv.fr/produits/grist
  - facile à déployer/dockeriser
  - ergonomie excellente (tableur + fiches/formulaires personnalisables)
  - format SQLite
  - dimension collaborative
  - extensible via des plugins JavaScipt (en cours de développement chez nous : un plug-in pour indexer des données Grist avec Opentheso)
  - possibilité de conférer deux noms aux colonnes : un pour les yeux de l'utilisateur, un autre pour l'API, qui sert naturellement de lieu où réaliser le mapping
  - API REST complète (mais documentation meh)
- Des couples efficaces pour transformer les données : Python + RDFLib, Deno + rdf.js, Rust + rdf.rs.

EXPLORER

[l’exemple de l’application web sherlock]

contexte

- Programme de recherche SHERLOCK à l'IReMus :
  - « Comment et pourquoi modéliser les données musicologiques avec le CIDOC CRM ?»
  - « Comment faire interagir les données sémantiques et les sources ? »
  - « Comment publier et manipuler les données sémantiques ? »
- Pas ou peu d'apport financier.
- Développeur (presque) unique.
- Recourir à un modèle unique dans les différents projets permet de ne concevoir, développer et maintenir qu'une unique application pour présenter et exploiter les données.
- Technologies :
  - Front : TypeScript, React, Hero UI, Tailwind CSS
  - Back : Apache Jena Fuseki, Docker, Traefik

PÉRENNISER

[discussion]